Computation for Analytics

DIANE WOODBRIDGE, PH.D

Contents

Python File I/O

Function

Data Aliasing

pytest

Contents

Python File I/O

Function

Data Aliasing

pytest

Python File I/O

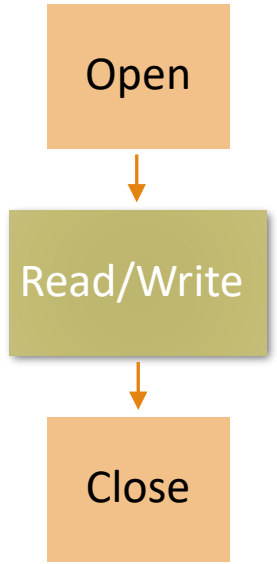
As a data scientist, we have various sources to access data including downloading files, calling APIs, accessing databases, etc. You'll learn more of those skills in the Fall module, but let's practice how to read files that you obtained and write processed outputs.

- To **read**, you need to 1) open a file, 2) either read (load) the entire file or line by line onto the memory, and 3) close the file when finish reading.
- To **write** , you need to 1) open a file, 2) either write output to the file, and 3) close the file when finish writing.

https://docs.python.org/3/tutorial/inputoutput.html#reading-and-writing-files

Python File I/O





https://docs.python.org/3/tutorial/inputoutput.html#reading-and-writing-files

Python File I/O

Step 1. Opening a file

- `open(filepath, mode='r', encoding=None)`
  - `filepath` - file path including the file name
  - `mode` - 'r' for reading, `'w'` for writing, `'a'` for appending, `'r+'` for reading and writing
  - encoding - You can specify, if the file has certain encoding including ASCII (`"ascii"`), UTF-8 (`"utf-8"`), etc.

- Example : `f = open('workfile', 'w', encoding="utf-8")`

Python File I/O

Step 2-1. Reading a file

- `f.read(size)` : Read "`size`", the number of bytes of data. If size is omitted, the entire file will be read.
- `f.readline()` : Read a single line from the file object, f.
- `f.readlines()` : Return a list of remaining lines of the file object, f.

Python File I/O

Step 2-1. Writing to a file

- `f.write(string)` : Write the contents of string to the file object, f.

Python File I/O

Step 3. Close the file

- `f.close()` : close the file object and immediately free up any system resources. If you do not close, it can take up more memory and slow down the process. Also, it may not write edits to the file.

- To avoid this issue, you can use

```
with open('workfile') as f:
```

and the file is properly closed after finishes. This is **recommended** way of opening a file.

Output Formatting

1. Formatted String Literals
   - Begin a string with **f** or **F** before an opening quotation mark or triple quotation mark and use python expressions within {} in side of quotation mark.
   - For numbers, you can format further using `:[flags][width][.precision]type`
     - `ex. f'''site {site_id:08.3f} is converted'''`
2. String format() Method
   - Use brackets and keyword/positional arguments within them and pass variables with **.format()** method. The positional argument's position refers to one in .format().
     - `ex. 'site {0:08.03f} is converted'.format(site_id)`
3. Manual String Formatting
   - You can use string concatenation to format as well.

ex-string_formatting.ipynb

https://python-reference.readthedocs.io/en/latest/docs/str/formatting.html
https://docs.python.org/3/tutorial/inputoutput.html#fancier-output-formatting

Example 1

From epa_air_quality.csv which includes date, site_id, pm10, and aqi values, return ‘date, site_id : pm10’ for records where pm10 is greater than 200.

- Extra : Sort the output by pm10 in descending order.
- Expected Output:

```
2020-02-02, 60270022 : 562
2020-02-02, 60270025 : 537
2020-02-02, 60270030 : 361
2020-02-02, 60271003 : 253
2020-08-22, 60190500 : 212
2020-02-02, 60270028 : 212
2020-08-21, 60190500 : 211
```

Contents

Python File I/O

Function

Data Aliasing

pytest

Functions

A block of code that can be called with parameters and can return data.

Syntax

- **Creating a function :** You can also add type annotation for your input/output.

```
def function_name(args):
    return value1, value2, …
```

```
def function_name(args: type = default_val) -> type:
    return value1, value2, …
```

  - ***args***
    1. Keyword Arguments separated by commas
       - `function_name(arg1: type = val1, arg2: type = val2)) -> type`
    2. Arbitrary Arguments, *args - The function receives a **tuple** of arguments
       - `function_name(*args: type) -> type`
    3. Arbitrary Keyword Arguments, **kwargs - The function will receive a dictionary of arguments
       - `function_name(**kargs: type) -> type`
- **Calling a function**

```
val1, val2, … = function_name(args_values)
```

https://docs.python.org/3/library/typing.html
https://peps.python.org/pep-0484/#arbitrary-argument-lists-and-default-argument-values

Functions

- main() function
  - In many programming languages including C++/Java, the main function is the entry point of the program. However, Python doesn't require main() function and it starts from the first line outside of the main() function. This reduce readability and means that executing your .py might execute whatever written in the file.
    - Solution
      - Use a special keywords called `__name__` which returns the name of the current module.
      - If it is a top-level of the program, it will return '`__main__`'
    - Good to define main() function.
      - Ex.

```
def main():
    statements

if __name__=='__main__':
    main()
```

https://docs.python.org/3/library/__main__.html

Example 2

For the following functions, what are the proper ways to call the function?

```
def generate_emails_keyword(first_name, last_name):
    user_id = first_name.lower()[0] + last_name.lower()
    return user_id + '@usfca.edu'
```

```
def generate_emails_arbitary(*args):
    user_id = args[0].lower()[0] + args[1].lower()
    return user_id + "@usfca.edu"
```

```
def generate_emails_arbitary_keyword(**kargs):
    user_id = kargs['first_name'][0].lower() + kargs['last_name'].lower()
    return user_id + "@usfca.edu"
```

Example 3

For a given sentence (input), create a function called find_palindrome_words() takes a string and return a list of strings that are palindrome words. A palindrome word remains the same when read forwards and backwards.

Ex. input_sentence = 'I did not like to listen a tot pop at Noon'

```
find_palindrome_words(input_sentence) == ['I', 'did', 'a', 'tot', 'pop', 'Noon']
```

Functions

Decorator

- You can extend functions by passing function to another function using `@function_name` in front of another function - decorator takes function as input and returns another function or object.
  - It is a syntactic sugar - Not necessary to have but makes things easier.
- Good for improving readability and easily extending functionality.
  - You will use this to build a Flask web application in Data Acquisition and Data Science Entrepreneurship classes.
    - Ex. `@login_required` in Flask: Only allows authenticated users can access the function.

ex-decorator.ipynb

https://flask-login.readthedocs.io/en/latest/#flask_login.login_required
https://peps.python.org/pep-0318/
https://wiki.python.org/moin/Decorators

Functions

Decorator

- Definition

```
def decorator_func(f):
      def inner_function(*args): # this takes arguments from f
            statements
      return inner_function
```

- Usage

```
@decorator_func
def func():
    statements
```

  - Calling `func` is equivalent to `decorator_func(func)`.

ex-decorator.ipynb

https://wiki.python.org/moin/Decorators
https://peps.python.org/pep-0318/

Example 4

return_wage() is a function that takes a string as input and returns as float.

```
def return_wage(val):
    return float(val)
```

Let’s create a decorator called ten_percent_increase so that we can easily return 10% increased value of the original input.

- Ex. return_wage(‘100') should return 110

Contents

Python File I/O

Function

Data Aliasing

pytest

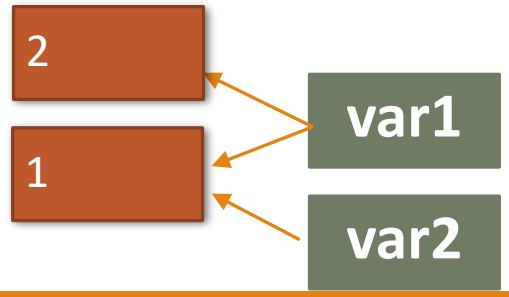
Data Aliasing

Data Alias

- Provide another name (var2) to the existing data.

```
var1 = 1
var2 = var1
```

  - This is basically referring the same data by two different names. : Remember that `var1` and `var2` are variable names and the actual data is 1, an integer value.
  - To check, you can use `id(var_name)` which returns the memory address of `var_name`



ex-data_aliasing.ipynb

Data Aliasing

Passing arguments to function

- Aliasing of data also happens when we pass variables to functions, which can change the original variable if it is a mutable type.
  - That's why it is **important to choose immutable/mutable types** to avoid unintended changes.

ex-data_aliasing.ipynb

https://realpython.com/python-mutable-vs-immutable-types/#common-mutability-related-gotchas

Example 5

After completing each year, employees earn certain increase from the last year's salary.

- For instance, increase_scale_sequence = [0.01, 0.02, 0.03] means that on one's second year the person will get 1% increase from the base salary. On the third year, one will get 2% increase from the second year.
- One developed a function called yearly_salary() as below.
  What is the mistake the developer made and how would you fix it?

```
def yearly_salary(increase_scale_sequence, year, base):
    for i in range(len(increase_scale_sequence)):
        if i == 0:
            increase_scale_sequence[i] = increase_scale_sequence[i] + 1
        else:
            increase_scale_sequence[i] = increase_scale_sequence[i-1] \
                                       * (increase_scale_sequence[i] + 1)
    return increase_scale_sequence[year-2] * base
```

Contents

Python File I/O

Function

Data Aliasing

pytest

Unit Test

Automated tests that the section (class, module, function, etc.) of your code behaves as intended.

Good practices

- Use descriptive names for testing functions.
- A testing unit should focus on a small bit of functionality and prove it works.
- Each test unit should be independent.
- Always run the full test suite before a coding session, and run it again after. - Never push code to the repository, if it fails any unit tests.

https://docs.python-guide.org/writing/tests/

pytest

Python-based unit test framework.

Installation

- `$ conda install pytest`

```
(Computation_Bootcamp) ML-ITS-901885:Class dwoodbridge$ conda install pytest
Collecting package metadata (current_repodata.json): done
Solving environment: done

## Package Plan ##

  environment location: /Users/dwoodbridge/opt/anaconda3/envs/Computation_Bootcamp

  added / updated specs:
    - pytest


The following packages will be downloaded:

    package                    |            build
    ---------------------------|-----------------
    openssl-1.1.1q             |       hca72f7f_0         2.2 MB
    pytest-7.1.2               |   py39hecd8cb5_0         443 KB
    tomli-2.0.1                |   py39hecd8cb5_0          25 KB
    ------------------------------------------------------------
                                           Total:         2.7 MB
```

https://docs.pytest.org/en/7.1.x/

pytest

You can use the `assert` statement to verify test expectations.

- The `assert` keyword lets you test if a condition in your code returns True, if not, the program will raise an AssertionError.

test_count_min_max.py

```
from count_min_max import *

def test_count_min_max():
    assert count_min_max([5,4,3]) == (3, 3, 5)
```

https://docs.pytest.org/en/latest/

pytest

Run pytest

To run multiple test files, `$pytest` will run all files of the form test_*.py or *_test.py in the current directory and its subdirectories.

- If you don't have "test" in your .py file, you can directly call `$pytest your_code.py`

```
bash-3.2$ pytest
============================= test session starts ==============================
platform darwin -- Python 3.9.12, pytest-7.1.2, pluggy-1.0.0
rootdir: /Users/dwoodbridge/Class/2022_MSDS501/Example/Day4
plugins: anyio-3.5.0
collected 1 item

test_count_min_max.py .                                                  [100%]

============================== 1 passed in 0.05s ===============================
```

pytest

To run multiple test files, pytest will run all files of the form test_*.py or *_test.py in the current directory and its subdirectories.

- If you don't have "test" in your .py file, you can directly call pytest your_code.py

```
bash-3.2$ pytest
============================= test session starts ==============================
platform darwin -- Python 3.9.12, pytest-7.1.2, pluggy-1.0.0
rootdir: /Users/dwoodbridge/Class/2022_MSDS501/Example/Day4
plugins: anyio-3.5.0
collected 1 item

test_count_min_max.py F                                                  [100%]

=================================== FAILURES ===================================
_____________________________ test_count_min_max ______________________________

    def test_count_min_max():
>       assert count_min_max([5,4,3]) == (3, 3, 5)
E       NameError: name 'count_min_max' is not defined

test_count_min_max.py:4: NameError
=========================== short test summary info ============================
FAILED test_count_min_max.py::test_count_min_max - NameError: name 'count_min_...
```

Contents

Python File I/O

Function

Data Aliasing

pytest

References

Python : https://docs.python.org/3/

Pytest : https://docs.pytest.org/en/7.1.x/